

	Departamento: DCCO
	Carrera: Ingeniería en Software Asignatura: Análisis y diseño de software NRC:30724
	Apellidos y nombres del estudiante: • Lasso Sebastián • Valencia Yuliana • Villarroel Justin

ANTECEDENTES

1. La práctica se fundamenta en la necesidad de transformar requisitos funcionales de **nivel 0** en casos de uso que sean claros, trazables y verificables para el analista.
2. Se utiliza un **CRUD de Estudiante** como caso base, el cual permite gestionar información crítica como **ID, Nombre y Edad**, sirviendo como punto de partida para el análisis estructural y dinámico

OBJETIVO

1. Analizar el comportamiento de un sistema CRUD de Estudiantes mediante la integración de diagramas de casos de uso, de clases y de secuencia, con el fin de definir y comprender las responsabilidades específicas de cada elemento dentro de la arquitectura.
2. Desarrollar de manera integral el análisis funcional, estructural y dinámico de un sistema CRUD de estudiantes utilizando PlantUML para garantizar la trazabilidad entre requisitos y modelos de análisis.

DESARROLLO

Modelado de casos de uso de nivel 0 y apoyo al análisis orientado a objetos con PlantUML

Requisitos funcionales de nivel 0 para el CRUD

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir agregar un nuevo estudiante con ID, Nombre y Edad.	Administrador académico	Agregar estudiante
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente usando ID, Nombre y Edad.	Administrador académico	Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante seleccionado cuando corresponda según la regla académica definida.	Administrador académico	Eliminar estudiante
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados en una tabla con las columnas ID, Nombre y Edad.	Administrador académico	Mostrar todo

RF-05	El sistema debe validar los datos obligatorios antes de guardar o actualizar cambios.	Sistema	Validar datos del estudiante
-------	---	---------	------------------------------

Código PlantUML: diagrama de casos de uso CRUD Estudiante

```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema CRUD de Estudiantes" {
    usecase "Agregar estudiante" as UC1
    usecase "Actualizar estudiante" as UC2
    usecase "Eliminar estudiante" as UC3
    usecase "Mostrar todo" as UC4
    usecase "Validar datos del estudiante" as UC5
}

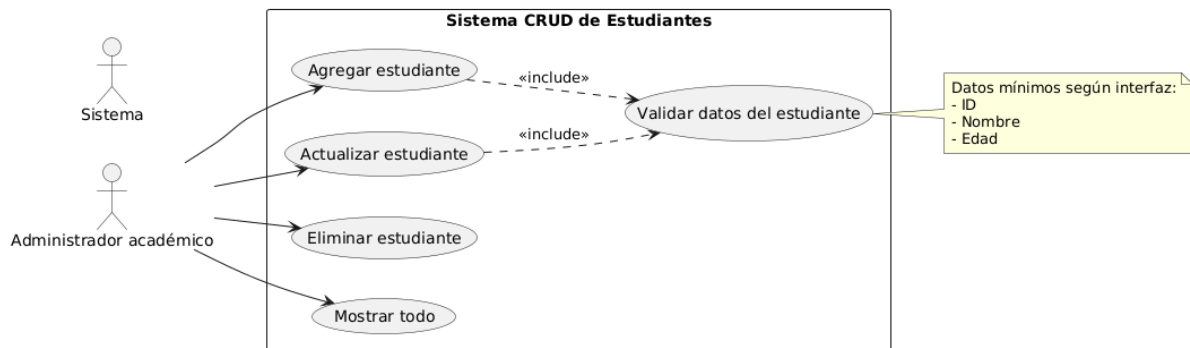
Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>

note right of UC5
    Datos mínimos según interfaz:
    - ID
    - Nombre
    - Edad
end note
@enduml

```

Representación

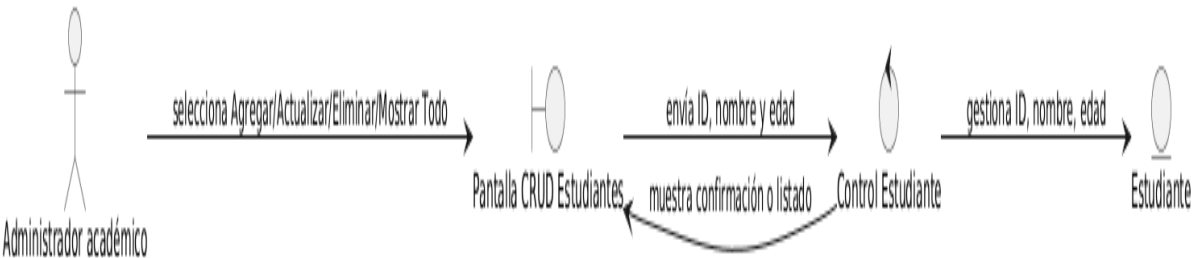


Puente entre casos de uso y clases de análisis

```
@startuml
left to right direction

actor "Administrador académico" as Admin
boundary "Pantalla CRUD Estudiantes" as UI
control "Control Estudiante" as Ctrl
entity "Estudiante" as Est

Admin --> UI : selecciona Agregar/Actualizar/Eliminar/Mostrar Todo
UI --> Ctrl : envía ID, nombre y edad
Ctrl --> Est : gestiona ID, nombre, edad
Ctrl --> UI : muestra confirmación o listado
@enduml
```



Matriz de trazabilidad RF-CU para el ejemplo

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agregar estudiante	Administrador académico	Alta	El sistema guarda un estudiante con ID único, Nombre obligatorio y Edad válida.	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema modifica Nombre y/o Edad luego de validar campos obligatorios.	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante seleccionado.	UC3

RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con las columnas ID, Nombre y Edad.	UC4
RF-05	Validar datos del estudiante	Sistema	Alta	El sistema impide guardar o actualizar registros con ID vacío, nombre vacío o edad inválida.	UC5

Relación entre diagrama de clases de análisis y diagramas de secuencia de análisis

Derivación de clases de análisis

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del actor y muestra resultados.	FormularioCrudEstudiante	Representa la pantalla con campos ID, Nombre, Edad y botones Agregar, Actualizar, Eliminar, Mostrar Todo.
Control	Coordina la lógica de cada caso de uso.	ControlEstudiante	Valida datos, decide si agrega, actualiza, elimina o consulta la lista.
Entity / Entidad	Representa la información del dominio.	Estudiante	Contiene los atributos id, nombre y edad.
Repositorio / Colección	Conserva y recupera registros.	RepositorioEstudiante	Permite guardar, buscar, actualizar, eliminar y listar estudiantes.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioCrudEstudiante" <<boundary>> {
- txtId: String
- txtNombre: String
- txtEdad: String
+ clickAgregar()
+ clickActualizar()
+ clickEliminar()
+ clickMostrarTodo()
+ mostrarMensaje(mensaje: String)
+ mostrarTabla(estudiantes: List<Estudiante>)
}
```

```

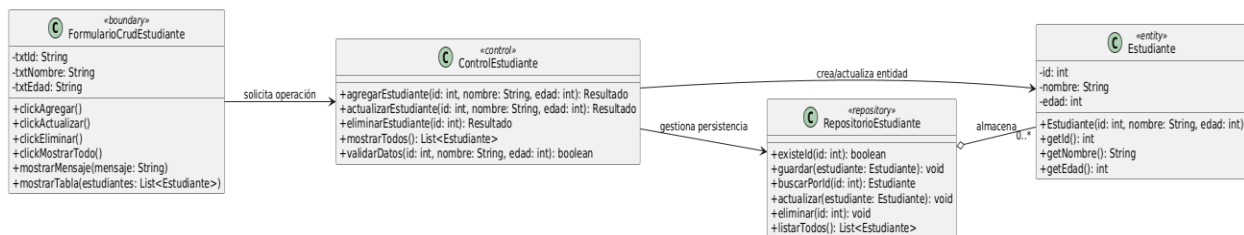
class "ControlEstudiante" <<control>> {
+ agregarEstudiante(id: String, nombre: String, edad: int): Resultado
+ actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
+ eliminarEstudiante(id: String): Resultado
+ mostrarTodos(): List<Estudiante>
+ validarDatos(id: String, nombre: String, edad: int): boolean
}

class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}

class "RepositorioEstudiante" <<repository>> {
+ existeId(id: String): boolean
+ guardar(estudiante: Estudiante): void
+ buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: String): void
+ listarTodos(): List<Estudiante>
}

```

FormularioCrudEstudiante --> ControlEstudiante : solicita operación
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena
@enduml



Diagramas de secuencia de análisis

Secuencia: Agregar estudiante

```

@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

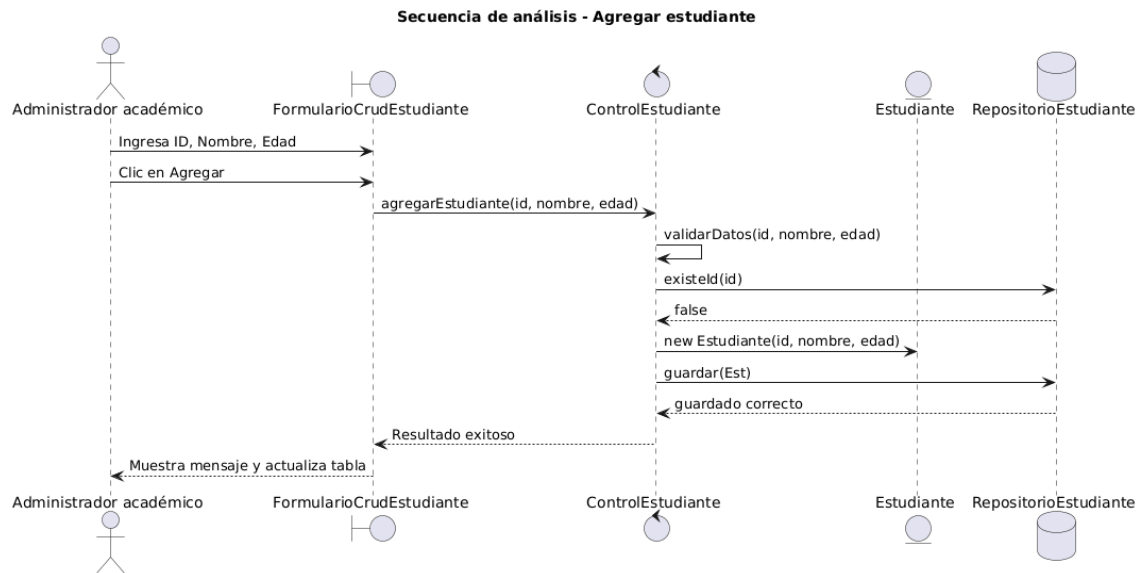
Admin -> UI : Ingresar ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso

```

```

UI --> Admin : Muestra mensaje y actualiza tabla
@enduml

```



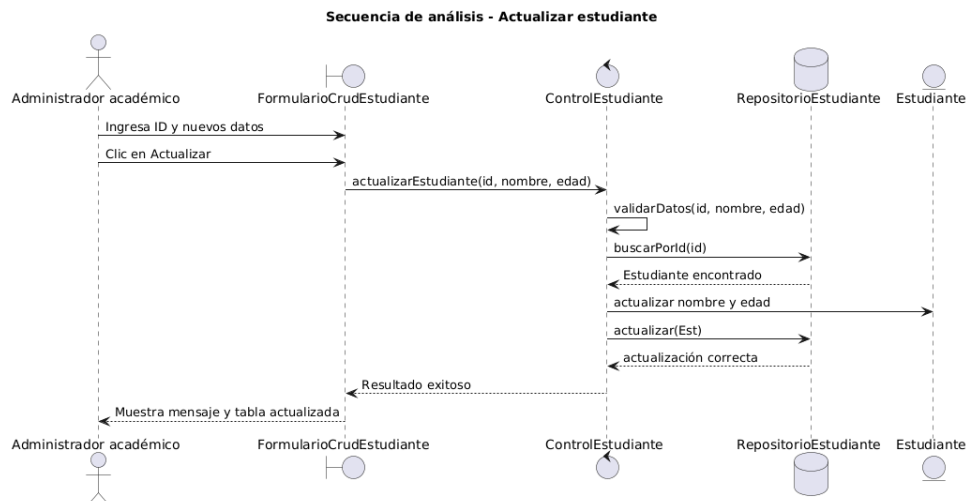
Secuencia: Actualizar estudiante

```

@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

Admin ->> UI : Ingresar ID y nuevos datos
Admin ->> UI : Clic en Actualizar
UI ->> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl ->> Ctrl : validarDatos(id, nombre, edad)
Ctrl ->> Repo : buscarPorId(id)
Repo -->> Ctrl : Estudiante encontrado
Ctrl ->> Est : actualizar nombre y edad
Ctrl ->> Repo : actualizar(Est)
Repo -->> Ctrl : actualización correcta
Ctrl -->> UI : Resultado exitoso
UI -->> Admin : Muestra mensaje y tabla actualizada
@enduml

```

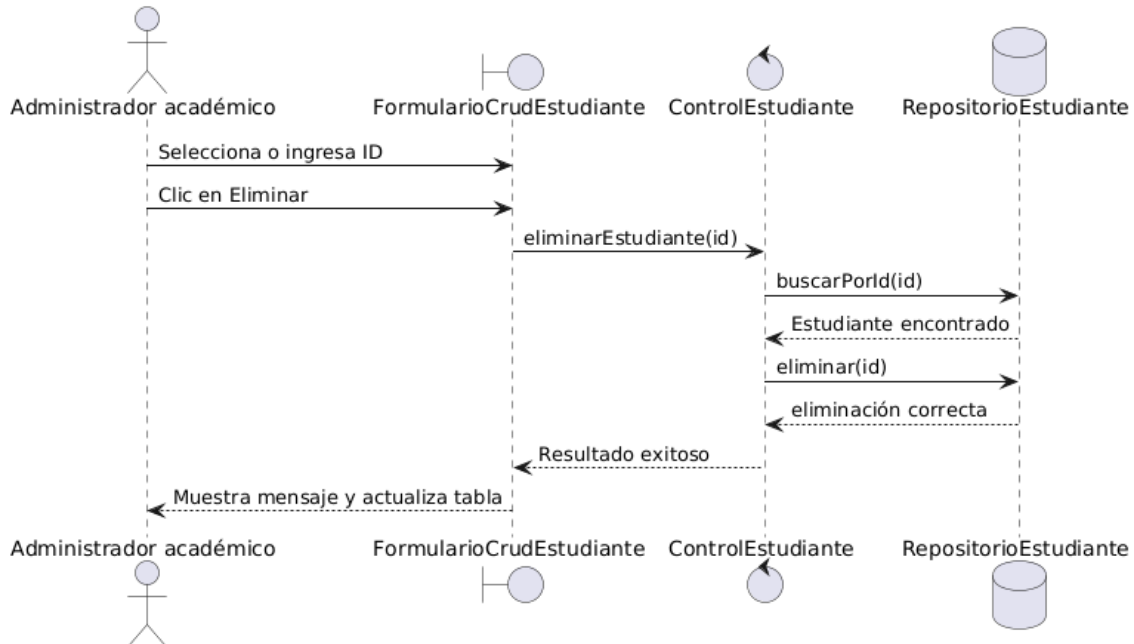


Secuencia: Eliminar estudiante

```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
```

```
Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

Secuencia de análisis - Eliminar estudiante

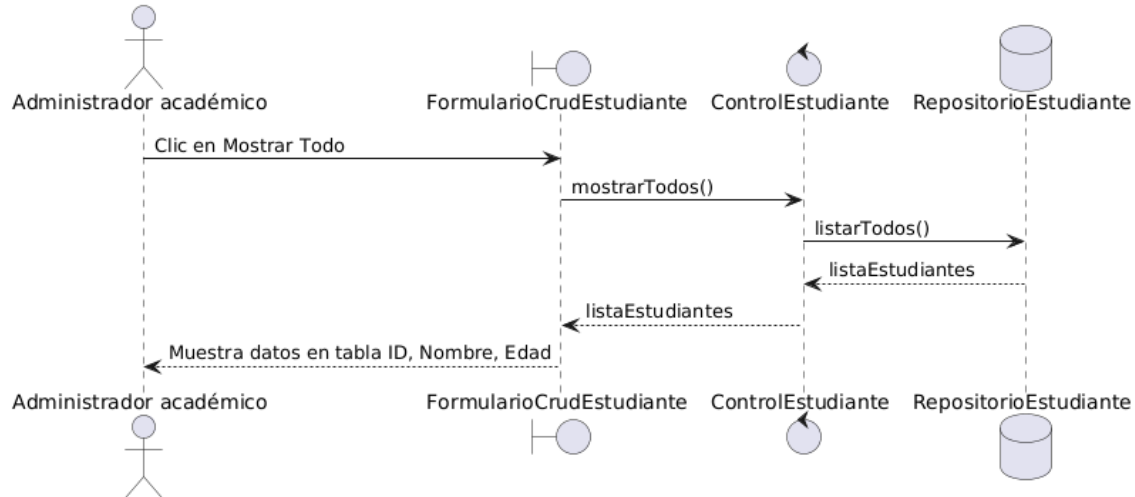


Secuencia: Mostrar todos los estudiantes

```
@startuml
title Secuencia de análisis - Mostrar todos los estudiantes
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
```

```
Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla ID, Nombre, Edad
@enduml
```

Secuencia de análisis - Mostrar todos los estudiantes



Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Agregar estudiante	Captura ID, Nombre y Edad desde el formulario.	Valida datos, verifica ID único y coordina el guardado.	Crea Estudiante y lo guarda en RepositorioEstudiante.	Secuencia: ingresar datos -> validar -> verificar ID -> crear entidad -> guardar -> mensaje.
Actualizar estudiante	Captura ID y nuevos valores.	Valida datos, busca existencia y coordina actualización.	Actualiza Estudiante existente en RepositorioEstudiante.	Secuencia: ingresar cambios -> validar -> buscar -> actualizar -> mensaje.
Eliminar estudiante	Solicita ID o selección de fila.	Verifica existencia y confirma eliminación.	Elimina el registro del RepositorioEstudiante.	Secuencia: seleccionar -> confirmar -> eliminar -> notificar.
Mostrar todos los estudiantes	Solicita mostrar lista y presenta tabla.	Solicita todos los registros.	RepositorioEstudiante devuelve lista de Estudiante.	Secuencia: solicitar listado -> recuperar lista -> mostrar tabla.

CONCLUSIONES

1. La implementación de una matriz de trazabilidad RF-CU es fundamental para garantizar que cada funcionalidad del sistema (CRUD de Estudiantes) esté directamente vinculada a un requisito funcional de nivel 0, lo que asegura que el modelo sea consistente y no existan omisiones en el análisis. Asimismo, el uso de PlantUML para el modelado de casos de uso permite definir con precisión los límites del sistema y la participación de actores externos (como el Administrador Académico), facilitando una transición técnica clara hacia el modelado estructural y dinámico sin pérdida de información. Finalmente, la modelación de validaciones mediante la relación «include» permite identificar responsabilidades obligatorias del sistema antes de la persistencia de datos, mejorando la calidad del análisis orientado a objetos.
2. La estructuración del sistema CRUD de Estudiantes evidencia una aplicación del patrón Modelo-Vista-Controlador (MVC), donde se delegan responsabilidades estrictas a la interfaz, la lógica de coordinación y la persistencia de datos. Esta separación de intereses, definida estáticamente en el diagrama de clases, se valida de forma dinámica mediante los diagramas de secuencia, los cuales confirman que la interfaz nunca interactúa directamente con la base de datos sin antes ser procesada por una capa de control. En consecuencia, la sincronización entre ambos diagramas garantiza la viabilidad operativa del modelo. Al asegurar que todo flujo de eventos iniciado por el administrador académico sea realizado de manera

centralizada para validar, buscar o modificar entidades, se establece una arquitectura robusta y de bajo acoplamiento que mitiga errores de diseño antes de la etapa de implementación.

RECOMENDACIONES

1. Se recomienda mantener un rigor estricto en la definición de los tipos de datos a lo largo de todos los artefactos de modelado. Es un error común en la fase de análisis declarar atributos estrictamente numéricos como el id como cadenas de texto (String) por simple facilidad de representación. Para evitar ambigüedades, las clases de tipo Entidad y Control deben reflejar siempre el tipo de dato lógico y nativo del dominio (por ejemplo, int para valores numéricos).

Sangolquí, a 28 de abril de 2026